The ZEUS agent building tool-kit

J C Collis, D T Ndumu, H S Nwana and L C Lee

There is an emerging desire among agent researchers to move away from developing point solutions to point problems in favour of developing methodologies and tool-kits for building distributed multi-agent systems. This philosophy has led to the development of the ZEUS agent building tool-kit, which facilitates the engineering of collaborative agent applications through the provision of a library of agent-level components and an environment to support the agent building process. The ZEUS tool-kit is a synthesis of established agent technologies with some novel solutions that provide an integrated environment for rapid software engineering of collaborative agent applications.

1. Introduction

Ideally developers should spend less time worrying about the intricacies of a particular technology, and more time implementing solutions to their problems. So as new technologies mature, generic application-independent solution libraries tend to be developed to aid the users of the technology — note, for example, the proliferation of class libraries of user-customisable components that are now available for users of the Java programming language. With the provision of well-engineered and relatively standardised class libraries, the development times for new systems that exploit such libraries tend to fall.

Agent technology is no exception to this general principle, and there is an emerging consensus among agent researchers of the need to develop methodologies and tool-kits for building distributed agent systems [1]. As the libraries of programming languages tend not to include the high-level constructs and concepts needed for agent applications, many tool-kits and frameworks of varying degrees of sophistication have recently been created to aid agent development; some examples are considered later in this paper.

The philosophy of moving away from point solutions to general frameworks, architectures and methodologies was the motivation for developing the ZEUS agent building tool-kit. The intention has been to facilitate the engineering of future agent applications by providing a library of agent-level components, together with an agent development methodology that describes how agents can be built using the low-level components from the library.

There are, however, many different types of agent. The ZEUS tool-kit was developed to assist the creation of collaborative agent systems — systems comprising a community of agents that work together to achieve shared objectives [2]. Section 2 describes the characteristics of collaborative agents, and the reasons why they are difficult to implement, while section 3 describes the components that the ZEUS tool-kit provides to build collaborative agents, and explains how the tool-kit's agent-building software and methodology supports the development process. Finally in section 4, discussion covers other related work on creating frameworks, methodologies and tool-kits for constructing collaborative agent systems.

2. What are collaborative agents?

There are many different types of agent of which information retrieval 'softbots' [3] and personal assistants [4] are perhaps the best known. Collaborative agents, which the ZEUS tool-kit builds, are typically large, coarse-grained agents that emphasise autonomy and co-operation with other agents. They generally work in open, time-constrained environments. They co-operate because each agent possesses limited competencies, information or resources, and thus, by pooling together their abilities, they are able to solve problems beyond the capability of any one single agent [2].

A typical example of a collaborative agent application is agent-based telecommunications network service provisioning (see, for example, Davison et al [5]). In such a scenario, each agent might be responsible for a small portion of a nation-wide network. For example, where a customer requests a high-fidelity connection between two nodes some distance apart on the network, agents need to co-operate, as there is no possible route within the jurisdiction of a single agent. Hence the agents along potential routes must negotiate among themselves, allocating bandwidth and evaluating what quality of service

is possible. This will lead to the formation of commitments, where agents agree to provide their local service at a certain time for a certain cost. Should an agent later withdraw from a commitment (say, through technical problems), the society will need to re-plan and form an alternative arrangement using a different route. This example illustrates some of the essential features of a collaborative agent (which are discussed in greater detail in Ndumu, Collis and Nwana [6]), namely that:

- they must be able to communicate — this is actually a challenging problem to solve, requiring not only a common transport protocol (like TCP/IP), but also a common inter-agent communication language and a shared ontology that represents the domain concepts being communicated,

- they must be able to reason — when control of a resource (like a service or a sum of money) is delegated to an agent, it needs sufficient intelligence to decide when to release the resource to potential users, or utilise the resource in pursuit of a goal; collaborative agents therefore need a means of determining how best to achieve their goals given their limited resources,

- they must be able to co-operate — because a single agent possesses limited competencies, resources or information, it will typically be capable of solving just a small portion of a larger problem, and so a collaborative agent faced with a problem it cannot solve alone must find other agents who do possess the requisite ability, negotiate with them, and ultimately delegate the task to them; this implies a means of co-ordinating the behaviour of different agents so that they act together in a coherent manner.

Since these requirements relate to the functionality of collaborative agents rather than any particular application, they are referred to as 'agent-level' issues, and they are not easy to satisfy [1]. Consequently, collaborative agents tend to be large, complex and difficult to implement. This high development overhead was one of the motivations behind the creation of the ZEUS agent building tool-kit, the subject of the next section.

3. The ZEUS tool-kit

The ZEUS tool-kit was motivated by the need to provide a generic, customisable, and scalable industrial-strength collaborative agent building tool-kit. The tool-kit itself is a package of classes implemented in the Java programming language, allowing it to run on a variety of hardware platforms. Java is an ideal language for developing multi-agent applications because it is object-oriented and multi-threaded, and each agent consists of many objects and several threads. Java also has the advantage of being portable across operating systems, as well as providing a rich set of class libraries that include excellent network communications facilities.

The classes of the ZEUS tool-kit can be categorised into three functional groups — an agent component library, an agent building tool and an agent visualisation tool (the main components of which are shown in Fig 1). Each of these three groups will be described in this section.

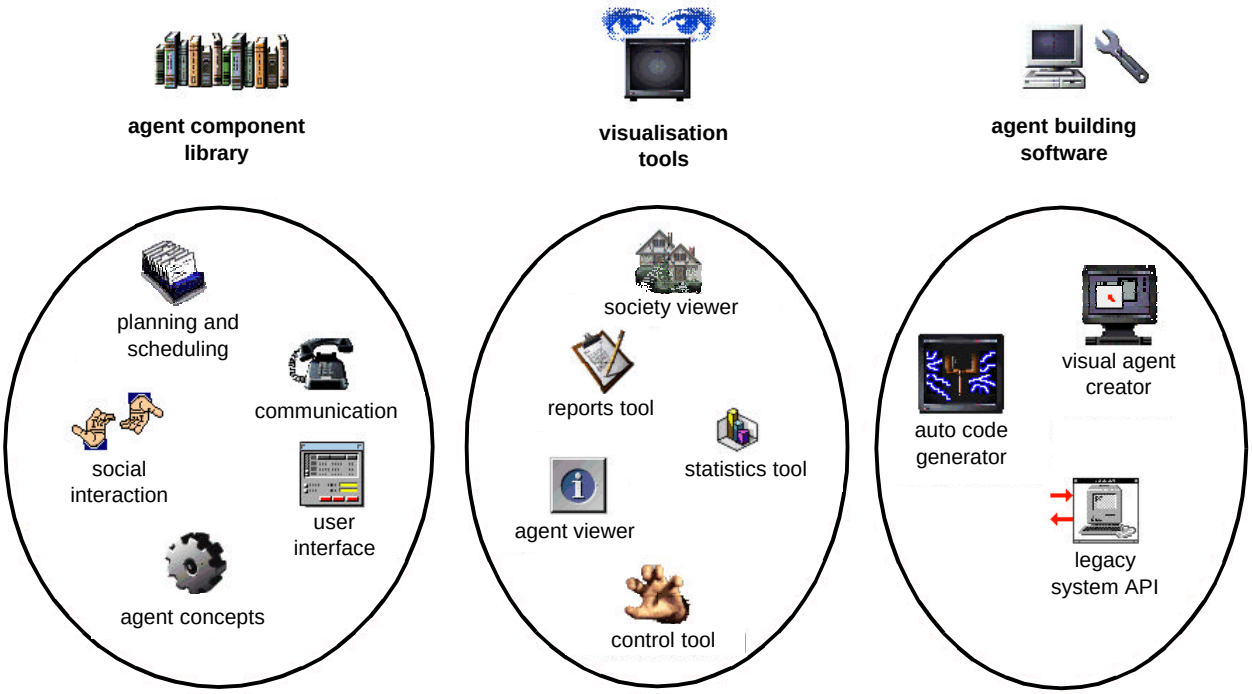

Fig 1 Components of the ZEUS agent building tool-kit.

3.1 The agent component library

The agent component library is a package of Java classes that form the 'building blocks' of individual agents. Together these classes implement the 'agent-level' functionality required for a collaborative agent. Thus for communication the tool-kit provides:

- a performative-based inter-agent communication language,
- knowledge representation and storage using ontologies,
- an asynchronous socket-based message-passing system.

In order to maximise future compatibility, the components of the ZEUS tool-kit utilise 'standardised' technology whenever possible; for instance, communication takes place through TCP/IP sockets using a language based on the Knowledge Query Management Language (KQML) [7]. In this spirit, it is planned that the Agent Communication Language that has recently been specified by the Foundation for Intelligent Physical Agents (FIPA) [8, 9] be adopted.

Next, to provide the agents with reasoning facilities, the tool-kit provides:

- a generic planning and scheduling system,
- an event model along with an application programming interface (API) that allows application programmers to monitor changes in the internal state of an agent, and externally control its behaviour.

To enable inter-agent co-operation, the tool-kit offers:

- a co-ordination engine that controls the behaviour of an agent — the functioning of the engine is influenced by the agent's knowledge context, e.g. the organisational relationships with other agents, available resources and competencies, available co-operation strategies.

For an agent to participate in goal-driven interactions with other agents its co-ordination engine needs one or more behavioural strategies and some idea of what relationships the agent has with each of its peers; hence the tool-kit also provides:

- a library of predefined co-ordination protocols,
- a library of predefined organisational relationships.

Together the components of the agent component library enable the construction of a generic application-independent ZEUS agent that can be customised for specific applications by imbuing it with problem-specific resources, competencies, information, organisational relationships and co-ordination protocols.

The component parts of such a generic agent, and the means by which it can be specialised for particular applications, are now described.

Logically, each ZEUS agent is composed of three layers — a definition layer, an organisational layer and a co-ordination layer. The definition layer comprises the agent's reasoning abilities, its goals, resources, skills, beliefs, preferences, etc. The organisation layer describes the agent's relationships with other agents, e.g. the agencies to which it belongs, what abilities it believes other agents possess, and what authority relationships exist between it and these other agents. At the co-ordination layer each agent is modelled as a social entity, i.e. in terms of the co-ordination and negotiation techniques it possesses. Built on top of the co-ordination layer are the protocols that implement inter-agent communication, while beneath the definition layer is the API that enables the agent to be linked to the external programs that provide it with resources and/or implement its skills.

Figure 2 shows how this logical agent model is realised using the classes of the ZEUS agent component library. The generic agent of Fig 2 includes the following components.

- Mailbox

  This handles communications between the agent and other external agents. It is a complex entity consisting of several other threads such as the server, which accepts and stores incoming messages.

- Message handler

  This processes incoming messages from the mailbox, dispatching them to other components within the agent.

- Co-ordination engine

  This makes decisions concerning the agent's goals, e.g. how they should be pursued, or when to abandon them. It is also responsible for co-ordinating the agent's overall activities with other agents using its known co-ordination strategies. This not only enables tasks to be contracted and delegated, but also ensures that agents jointly working towards a shared goal all behave in a coherent manner.

- Acquaintance model

  This describes the agent's relationships with other agents in the society, and its beliefs about the capabilities of those agents.

- Planner and scheduler

  This plans the agent's tasks based on decisions taken by the co-ordination engine and the resources and task definitions available to the agent.

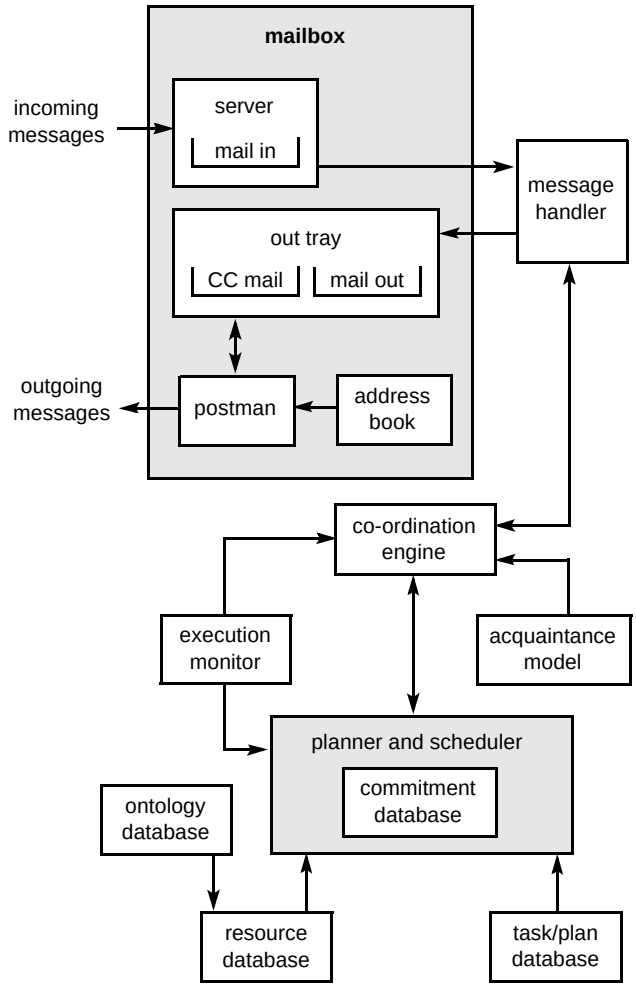

Fig 2 The flow of information between the components of a ZEUS agent.

- Resource database

  This maintains a list of resources (referred to in this paper as facts) that are owned and available to the agent.

- Ontology database

  This stores the logical definition of each fact type — its legal attributes, the range of legal values for each attribute, any constraints between attribute values, and any relationships between the attributes of the fact and other facts. Agents must use the same ontological information if they are to understand each other.

- Task/plan database

  This provides logical descriptions of planning operators (or tasks) known to the agent. It also stores a set of scripts that the agent uses to behave reactively to events.

- Execution monitor

  This starts, stops and monitors external systems that have been scheduled for execution or termination by the planner and scheduler. It also informs the co-ordination engine of successful and exceptional terminating conditions of the tasks it is monitoring.

These generic components act together to provide the necessary agent functionality. For instance, communication is facilitated by the mailbox and the ontology database; the former provides agents with the ability to transmit messages in a universally recognised format, while the latter enables each agent to understand what its correspondent is communicating. Once agents can communicate, interaction becomes possible, raising the prospect of sophisticated behaviour like tendering and negotiation. Such behaviour requires some degree of intelligence, and is provided in ZEUS agents by the planner and scheduler, and co-ordination engine components.

The agent library components are also used to implement standard ‘utility’ agents that are an important part of the ZEUS tool-kit. The utility agents fulfil a support role in their society and can be used in any application without modification. Firstly, the agent name server provides a ‘white pages’ service, matching agent names to network addresses just like the domain name servers of the Internet. Another utility agent, the facilitator, provides a ‘yellow pages’ service similar to the ‘Yahoo!’ index; it is used by agents looking for others who are capable of performing a particular task or service. The third type of utility agent, the visualiser, will be discussed later.

The next section describes an agent building methodology that helps users to create ZEUS-based application-specific agents.

3.2 The agent building software

The design philosophy of the ZEUS tool-kit was to delineate the sort of agent-level functionality described in the preceding section from domain-level problem-specific issues. In other words, the intention is to provide classes that implement communication, reasoning and co-operation, leaving developers to concentrate on defining the agents necessary for their application, and to provide the code that implements their agents’ particular domain-specific abilities.

Because so many aspects of conventional agent system development are handled by the ZEUS tool-kit, developers need a means of understanding what is expected of them. Consequently the ZEUS tool-kit was developed in parallel with an agent development methodology, the intention being that this methodology would guide users through the agent creation process. This section begins by outlining the methodology, and then describes how editors provided by ZEUS support the agent creation process.

The ZEUS agent development methodology

A methodology is a system of methods and principles that drive the conduct and products of some process. Methodologies are generally prescriptive in nature, not only facilitating the thinking and acting needed to perform the process, but also recommending methods and techniques that should be employed to achieve the process. The ZEUS agent design methodology is intended to hide most of the intricacies of multi-agent systems from developers, allowing them to concentrate on solving the problem in question. It has always been the intention that users of the ZEUS tool-kit need only be generally conversant with agent technology to create working agent applications, rather than having to be experts in distributed artificial intelligence.

The six stages of the ZEUS development methodology are shown in Fig 3. The stages are shown in the order they are attempted, with the developer moving down the activities concerned until all are completed (where stages are shown beside each other they can be attempted concurrently).

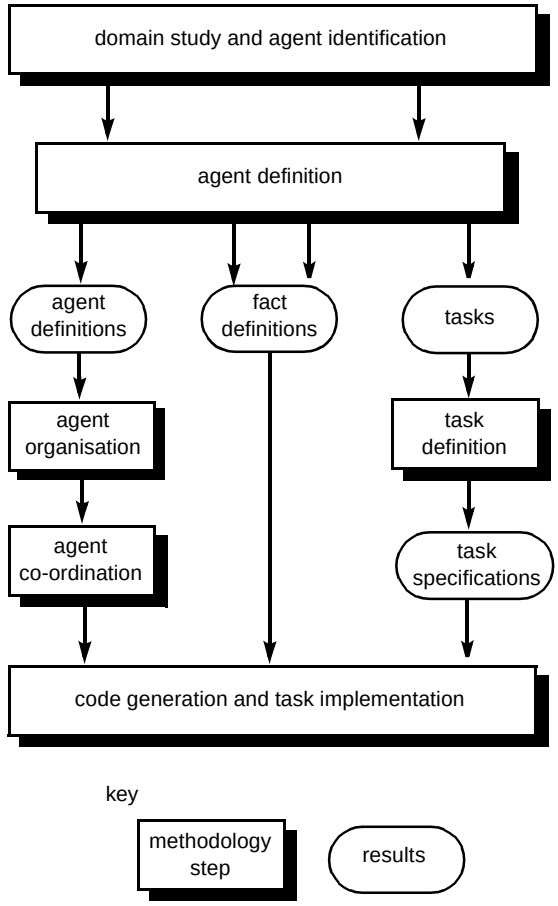

Fig 3 The stages of the ZEUS agent development methodology.

- The initial stage is domain study and agent identification, during which the developer analyses the problem domain to identify the potential agents. But what is an agent? This largely depends on the granularity at which the domain is modelled, but the best candidates will be those entities that are autonomous, i.e. mainly capable of functioning independently of human interaction, and those that are social, i.e. capable of interacting intelligently and constructively with other agents or humans. Entities that are responsive or proactive are also worth considering, as are those that are responsible for controlling a particular resource or service.

- Once the candidate agents are identified the agent definition phase can begin. This involves the developer identifying the significant attributes of each agent. One metaphor for an agent used at this stage of the methodology is that of factory manager who controls several production lines. Each agent begins life with a limited set of resources (called facts) and the ability to perform a number of different tasks simultaneously. The production of an item lasts for a finite period of time and consumes some amount of the agent's resources. Hence this stage involves the identification of the initial set of resources owned by the agent (and the definitions of each type), as well as the tasks the agent can perform.

- The agent definition phase continues until all agents have been considered, and their initial resources and tasks have been identified. The next stage is agent organisation where the developer identifies the acquaintances of each agent. To acquaint an agent *A* with another agent *B*, the resources that agent *A* believes agent *B* can produce are specified, along with the relationship agent *A* believes it shares with agent *B*. Agents that are defined as superior to other subordinate agents can delegate tasks to their subordinates. Agents can also belong to the same static 'community' and be co-workers who prefer to interact with one another. The default relationship is peer, which imposes no restrictions on interaction.

- As the organisation stage requires information about agents' abilities, it can be performed in conjunction with the task definition phase. During this stage, tasks that were identified during the agent definition phase are defined in terms of their costs, duration, preconditions, products and constraints. Tasks can be primitive (performed directly by executing a domain function), or summary (i.e. a complex task specified as a network of other tasks).

- When all tasks and acquaintances are specified, the agent co-ordination phase can begin. This involves identifying the appropriate co-ordination protocols each agent is likely to require for social interaction with other agents when performing its designated duties. Examples of such co-ordination protocols include master/slave for delegating tasks to

subordinates, contract net for contracting tasks out to peer agents, and various auction protocols for buying and/or selling resources. As the mechanics of these protocols have already been defined and included in the ZEUS tool-kit, the developer need only decide which is most applicable given the societal status and role of each agent.

- As the agent design methodology is supported by the agent generator software (described later in this section), the final stage of the methodology is code generation and task implementation. By this time the generator tool will have all the information necessary to automatically generate Java source code implementations for each agent. The generator also generates stub code for all external actions, of which task body implementations and database accesses are the best examples. As the individual agent source programs can be compiled into executable agents, the developer's sole task is to provide implementations for each task stub.

It is worth stating that the above description is only an outline of this agent creation methodology, which is the subject of a sizeable document in its own right. For instance, the initial domain study phase consists of several prescriptive sub-steps that are analogous to the analysis phase of the object-oriented programming methodology.

The ZEUS agent generator software

The ZEUS agent generator is a suite of integrated editors that support the ZEUS agent development methodology. To facilitate ease of use, the editors have been designed to enable users to interactively create agents by visually specifying their attributes. The current suite of editors includes:

- ontology — for defining the ontology items in a domain, i.e. the templates of legal facts, the legal attributes of these facts, the legal values for each attribute and any constraints between the fact's attributes,
- fact/variable — for describing specific instances of facts and variables, using the templates already created using the ontology editor,
- task description — for entering the attributes of tasks, and also enabling the composition of summary tasks, which consist of various sub-tasks ordered in some fashion,
- constraint — invoked by the task editor not only to define the restrictions (constraints) between the preconditions and effects of tasks, but it can also constrain the effects of a preceding task and the preconditions of a succeeding task in a summary task description (a typical constraint is along the lines of 'attempt to satisfy precondition *A* in parallel with *C*, but before *B*'),
- organisation — for defining the organisational relationships between agents,
- agent definition — for describing agents logically, which involves naming each agent's tasks and specifying their initial resources,
- co-ordination — for selecting the set of co-ordination protocols with which each agent will be equipped.

In order to generate the code for a specific application system, the generator tool inherits code from the agent component library, and integrates it with the data from the various visual editors. The resulting Java program code is then compiled and executed normally. Existing (legacy) systems can then be linked to the agents using the API of the (generic ZEUS-defined) wrapper class that wraps up the inherited code and the user-supplied data.

Used together, the agent component library and the agent building software facilitate the engineering of intelligent collaborative agent systems. The developer describes the intended agents with the agent creation tools, which generates the source code using classes from the component library. Once their tasks have been implemented the agents can be executed, and observed using the visualisation tools provided by the tool-kit; these are the subject of the next section.

3.3 The visualisation tools

An inherent problem of a distributed system like an agent society is attempting to visualise its behaviour. This is because the data, control and active processes are all distributed across the society. Consequently the analysis and debugging of multi-agent systems is difficult, as each agent has only a local view of the whole.

The onus is therefore on the user to integrate the large amounts of limited information generated by each agent into a coherent whole. The ZEUS tool-kit provides a utility agent called the visualiser that can be used to view, analyse or debug societies of ZEUS agents. The visualiser agent is built from the same fundamental components as every other agent in the society. It works by continually querying each agent about its states and processes, and then collating and interpreting the replies to create an up-to-date model of the agents' collective behaviour. This model can be viewed from different perspectives through the five visualisation tools, each of which emphasises a different aspect of the agent society:

- a society viewer shows all the agents and their organisational inter-relationships — it can also show the messages exchanged between the agents during problem solving,
- a reports tool shows the society-wide decomposition/ distribution of active tasks and the execution states of the various sub-tasks,

- an agent viewer enables the internal states of agents to be observed and monitored,
- a control tool is used to remotely review and/or modify the internal states of individual agents, thus enabling an agent's behaviour to be redefined at run-time by using this tool to modify its task, resource, or organisational databases, or even by redefining its co-ordination strategies — in this regard, the control tool is effectively an on-line version of the agent building software; it also facilitates administrative management, e.g. agents can be killed or suspended, and goals can be added or modified, etc,
- a statistics tool displays individual agent and society-wide statistics in a variety of formats.

The multi-perspective visualisation approach provided by the visualisation tools gives users the flexibility to choose what is visualised, how it is visualised and when it is visualised. The visualisation tools are generally used on-line, to visualise the interactions in a multi-agent society live, as they happen. However, the society, report and statistics tools can also operate off-line by recording agents' interaction sessions to a database. Once stored, recorded sessions can be replayed, video-recorder style, using the forward, rewind, fast-rewind and fast-forward buttons. Figure 4 is a screenshot showing three of the ZEUS visualiser windows (for a more detailed description of visualisation and debugging in ZEUS, see Ndumu et al [10]).

This section has described how low-level agent functionality is provided by the ZEUS tool-kit's agent component library, and how these components can be integrated into a fully functional collaborative agent. To build an agent more easily, developers can follow this methodology and use the agent building tools provided by the tool-kit. Finally, once the agents are implemented they

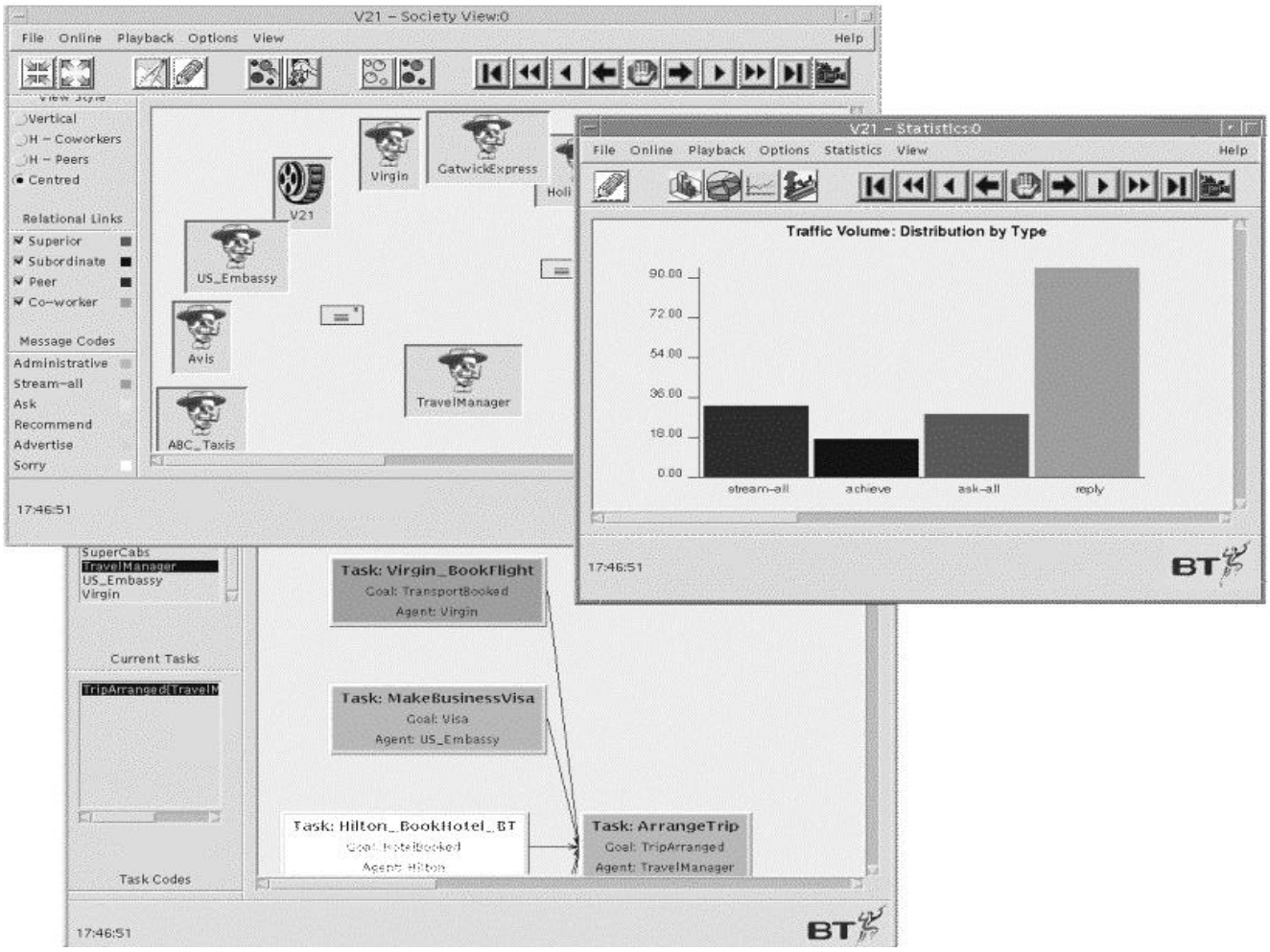

Fig 4 A screenshot of three of the visualisation windows. In the society viewer (top-left) each icon represents a single agent; the envelopes shown moving between agents signify the transmission of messages. The toolbar buttons on the top left of the society viewer window control the position and visibility of the agent icons, while the buttons on the top right control the 'video replay' facilities. The statistics tool (middle-right) shows a histogram analysis of the type of messages being exchanged during the current session. The reports tool (bottom) shows the names and states of the sub-tasks that form the task currently being attempted by a sub-group of the agents.

can be visualised, analysed or debugged using the visualisation tools provided.

Ndumu, Collis and Nwana [6] describe how the ZEUS tool-kit was used to develop a demonstration agent-based personal travel assistance system. In the demonstrator, a travel manager agent arranges a transatlantic trip on behalf of a user by interacting with hotel, airline, train and taxi reservation agents, and integrating the returned information to create a travel itinerary for the user. In addition, the ZEUS tool-kit has already been used to build prototype applications in the fields of network management, home shopping and electronic commerce [11].

4. Related work

There are several notable attempts at developing tool-kits for building agents of different functionalities and complexity. Some examples are the Java-based mobile agent frameworks that exploit the ease of creating mobile agents using the Java programming language; these include Aglets[1], Concordia[2] and Voyager[3]. Regarding collaborative agent development environments and tool-kits, there are a number of systems with similar general aims to ZEUS, e.g. dMARS [12], JAFMAS [13], KAoS [14, 15] and JAFIMA [16].

The dMARS architecture, developed at the Australian Artificial Intelligence Institute, has its conceptual underpinnings in the belief-desire-intention (BDI) model [17], and it provides an environment for developing distributed multi-agent systems. Thus, dMARS agents attempt to operationalise the BDI reasoning framework. Like ZEUS, the primary reasoning mechanism in dMARS agents is planning, with intentions operationalised as plans. (In dMARS, however, planning is more reactive than in the current version of ZEUS.) As there is less emphasis on co-ordination of multi-agent activity in dMARS, there is no explicit co-ordination engine (or equivalent), and it is unclear how general-purpose co-ordination protocols and strategies can be shared transparently among dMARS agents. The key non-functional difference between dMARS and ZEUS is possibly that of abstraction. It would appear that dMARS requires agent developers to work at a lower level of abstraction than that required by the ZEUS agent building environment.

JAFMAS (Java-based Agent Framework for Multi-Agent Systems) is primarily concerned with providing communication and co-ordination support for agent system developers. Co-ordination is supported through use of conversation classes that agents utilise to manage their interactions with other agents during problem solving. The conversation classes implement rule-based automata models, similar in spirit to the way co-ordination strategies are managed in ZEUS. Thus, a conversation class might implement the equivalent of the contract net co-ordination protocol, but specific to an agent and a given context. JAFMAS agents use broadcast communication to perform the services of the ZEUS facilitator agent, i.e. identifying agents with a particular capability.

KAoS is an ambitious project to devise an industrial-strength open agent architecture. One of its main distinctions from similar work is its explicit concern with security issues. Thus, in collaboration with Sun Microsystems, the KAoS design team are attempting to build security models that separate policy from mechanism. This way, agents may also negotiate about what security models to use in addition to negotiating about services. KAoS agents also provide conversation management support (as in JAFMAS) and an optional planner (as in ZEUS). KAoS, like ZEUS, also explicitly tries to raise the level of abstraction at which agent designers need to work to create agents. Thus, the KAoS agent design tool-kit supports mediating high-level representations for describing conversations and plans, and also intends to support one for describing security models [15].

JAFIMA (Java Framework for Intelligent and Mobile Agents) is a layered framework for the design of agent systems. It is designed and documented using patterns, and is also explicitly concerned with how the agents integrate with other system software. JAFIMA is perhaps one of the most comprehensive agent design frameworks, but in contrast to the tool-kit approach of say ZEUS or dMARS, JAFIMA is primarily targeted at developers wanting to develop agents from scratch. So while JAFIMA defines a complete development methodology, it does not provide pre-built agent-level components for use by developers.

5. Conclusions

This paper has described briefly the rationale, philosophy, underlying methodology and design of a tool-kit for constructing distributed collaborative agent applications. The creation of the ZEUS tool-kit was motivated by the considerable development overhead faced by those implementing distributed software systems. In this paper, it has been shown how employing a tool-kit can expedite development by supplying the functionality required for multi-agent systems.

By using the ZEUS tool-kit, agent developers can adopt the solutions to a wide range of technical challenges — these include agent communication languages, co-ordination, co-operation and negotiation, rational agent theory, visual programming, planning and scheduling, visualisation and knowledge representation using ontologies. It is hoped that, once freed from the intricacies of these technologies, developers will be able to devote their valuable time to the specifics of their problems, and so produce better applications.

[1] http://www.trl.ibm.co.jp/aglets/index.html
[2] http://www.meitca.com/HSL/Projects/Concordia/
[3] http://www.objectspace.com/voyager/

References

1 Ndumu D T and Nwana H S: 'Research and development challenges for agent-based systems', IEE Proceedings on Software Engineering, 144, No 1, pp 2—10 (1997), also available on-line from: http://www.labs.bt.com/projects/agents/

2 Nwana H S: 'Software agents: an overview', The Knowledge Engineering Review, 11, No 3, pp 205—244 (1996), also available on-line from: http://www.labs.bt.com/projects/agents/

3 Etzioni O and Weld D: 'A softbot-based interface to the Internet', Communications of the ACM, 37, No 7, pp 72—76 (1994).

4 Maes P: 'Agents that reduce work and information overload', Communications of the ACM, 37, No 7, pp 31—40 (1994).

5 Davison R G, Hardwicke J J and Cox M D J: 'Applying the agent paradigm to network management', BT Technol J, 16, No 3, pp 86—93 (July 1998).

6 Ndumu D T, Collis J C and Nwana H S: 'Towards desktop personal travel agents', BT Technol J, 16, No 3, pp 69—78 (July 1998).

7 Finin T and Labrou Y: 'KQML as an agent communication language', in Bradshaw J M (Ed): 'Software agents', MIT Press, Cambridge, Mass, pp 291—316 (1997).

8 The foundation for intelligent physical agents: 'The FIPA'97 Specification', available on-line from: http://drogo.cselt.it/fipa/spec/fipa97/fipa97.htm

9 O'Brien P D and Nicol R C: 'FIPA — towards a standard for intelligent agents', BT Technol J, 16, No 3, pp 51—59 (July 1998).

10 Ndumu D T, Nwana H S, Lee L C and Haynes H R: 'Visualisation and debugging of distributed multi-agent systems', to appear in Applied Artificial Intelligence.

11 Collis J C and Lee L C: 'Building Electronic Market-places with the ZEUS Tool-kit', Proceedings of the 1998 Agent Mediated Electronic Commerce (AMET-98) Workshop, Autonomous Agents '98, pp 17—32 (1998), available on-line from: http://www.labs.bt.com/projects/agents/

12 Ingrand F F, Georgeff M P and Rao A S: 'An architecture for real time reasoning and control', IEEE Expert, 7, No 6 (1992).

13 Chauhan D and Baker A D: 'JAFMAS: A multi-agent application development system', Proceedings Autonomous Agents '98, Minneapolis, MN, pp 100—107 (1998).

14 Bradshaw J M, Dutfield S, Benoit P and Woolley J D: 'KAoS: Toward an industrial strength open agent architecture', in Bradshaw J M (Ed): 'Software Agents', MIT Press, Cambridge, Mass, pp 375—418 (1997).

15 Bradshaw J M et al: 'Mediating representations for an agent design toolkit', KAW '98, Banff Canada (April 1998).

16 Kendall E A, Murali Krishna P V, Pathak C V and Suresh C B: 'An application framework for intelligent and mobile agent systems', to appear as chapter in Fayad M, Schmidt D C and Johnson R (Eds): 'Object Oriented Application Frameworks, Vol II', Wiley & Sons (1999).

17 Rao A S and Georgeff M P: 'BDI agents: from theory to practice', Proceedings First International Conference on Multi-Agent Systems (ICMAS-95), San Francisco, pp 312—319 (June 1995).

Jaron Collis obtained his BSc degree in Computation from the University of Manchester Institute of Science and Technology (UMIST), in 1993. In 1996 he completed his PhD at Queen's University, Belfast on the application of artificial intelligence to automatic scientific problem solving. He joined the intelligent systems research group at BT Laboratories in June 1997, and is currently working in the field of collaborative agents.

Divine Ndumu graduated in Civil Engineering from Imperial College in 1989, and Mathematics and Computing from the Open University in 1991. In 1989, he was winner of the Imperial College Governor's prize for excellence in Civil Engineering. In 1993 he was awarded a PhD in Engineering Applications of Artificial Intelligence from Imperial College. In 1992, he joined the University of Central Lancashire as a Lecturer, and in 1996/97 served a one year Research Fellowship secondment at BT Laboratories (BTL). In April 1997, Divine joined the Intelligent Systems Research Group at BTL, and currently leads the group's software agent research programme. In 1997, the ZEUS agent building tool-kit that he co-developed won the prestigious BCS IT award. The focus of his current research is on the practical applications of AI techniques to real-world problems.

Hyacinth S Nwana is a principal research scientist/engineer and technical group leader in the Applied Research and Technology (ART) department at BT Laboratories. He holds a BSc in Computer Science and Electronic Engineering, an MSc in Computer Science and a PhD in Artificial Intelligence/Computer Science (1988). He also recently completed (November 1997) an MEd degree in Computer Science Education at Queen's College, the University of Cambridge. He joined BTL in October 1995. In 1991, he principally won the DEC European AI prize. In 1997, he led BT's agents-based project (ABW-ZEUS) which won the prestigious British Computer Society top award for innovation. He is a member of the British Computer Society and a Chartered Engineer. He currently runs ART's Future Technologies group investigating novel biologically motivated computing models, software agents, believable interface agents, cognitive systems, and the application of such techniques to telecommunications and other computing problems.

Lyndon Lee received an MSc in Computer Science (Artificial Intelligence) with Distinction and a PhD in Computer Science from the University of Essex in 1989 and 1996 respectively. His doctoral thesis was in a formal model of multi-agent negotiation. From 1992-5 he was a recipient of the Croucher Foundation scholarship, and between 1987 and 1992 he worked as a developer and a consultant with a number of companies. After a short-term research fellowship with BT Laboratories (BTL), he joined the Intelligent Systems Research group in BTL in 1996, carrying out research and development of real-world multi-agent systems. In 1997, the ZEUS project that he co-developed in BT won the BCS IT award and since then has four patents on application. His research interests lie primarily in distributed artificial intelligence, artificial life, decision theory, game theory, and distributed and nonlinear systems. He is a member of BCS.